

Week 1 Lab - CS010A

TA: Sourabha Bharadwaj

Instructors: Prof. Kelly Downey and Prof. Saul Gonzalez

September 29, 2026

Agenda

- About me
- About Labs this quarter
- What is an algorithm?
- Writing your own code plan
- Quick recap of C++ code syntax
- Designing good test cases
- Questions
- The cutting edge

About me

- Sourabha Bharadwaj
 - Email: sbang032@ucr.edu
- PhD student in my 2nd year, working on computer networks
- Your TA for this quarter
- You can ask general questions in edstem so everyone can benefit
- Personal questions can also be addressed there to any TA
- Office hours:
 - In-person: WCH110
 - Online: Will share Zoom link (will confirm shortly)
 - Time: Mostly Fridays 2-4 pm (will confirm shortly)

About labs this quarter

- No need for any computers in this lab section
- Labs are graded out of 5 points
 - 1 point for being here on time
 - 1 point for completion and effort
 - 3 points for each correctly completed portion
 - Thorough code plan
 - Test cases
 - C++ code with no errors
- Form groups of 2
 - **Each person must complete the lab**
- Please submit your sheets before you leave so I may check
 - Please mention your Name and NetID (e.g. jsmith123)
- You may leave once you finish the in-person lab, but you are encouraged to stay and work on other course material

Lab 1: What you'll do?

1. Read assignment thoroughly
2. Write out a code plan
3. Write out clear test cases to test program
4. **Finally**, move on to writing C++ code

Before any of that, it is useful to understand what a basic algorithm is and how you write one.

What is an algorithm?

- A step-by-step set of instructions to solve a given problem
- It's how we break a big task into smaller actions
- Like a recipe, it is clear, logical, and the order in which you do things matters

Real-world example: Steps for frying an egg

1. Get a pan and put it on the stove
2. Turn on medium heat
3. Put oil into the pan
4. Crack egg into the pan
5. Wait 2-3 minutes for egg to fry
6. Flip and wait for 1 min
7. Transfer to a dish
8. Enjoy

What is an algorithm?

- A step-by-step set of instructions to solve a given problem
- It's how we break a big task into smaller actions
- Like a recipe, it is clear, logical, and the order in which you do things matters

Real-world example: Steps for frying an egg

1. Get a pan and put it on the stove
2. Turn on medium heat
3. Put oil into the pan
4. Crack egg into the pan
5. Wait 2-3 minutes for egg to fry
6. Flip and wait for 1 min
7. Transfer to a dish
8. Enjoy

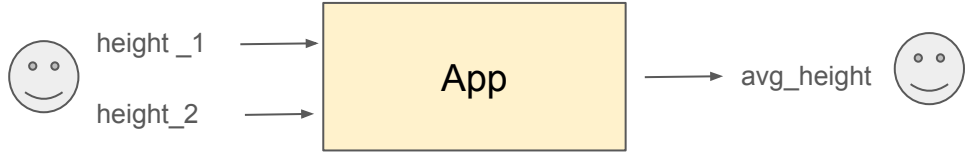
A code plan is no different!

- Basically the same as an algorithm
- Step-by-step outline of your process, in plain English
- Explains what your code should do, line by line, before you write in C++

Example of code plan

Problem: **Ask two users for their height and output the average**

Code Plan for the “App”:



1. Initialize variables (2 integer variables to hold heights, 1 double variable to hold the result)
2. Ask users for their heights, and store each height in corresponding variable
3. Compute the result as the average of their heights and store in the result variable
4. Print the result

Translating code plan to C++

Plain English: Initialize integer variable

C++: `int height;`

Plain English: Get height from user

C++: `cin >> height;`

Plain English: Arithmetic (calculate average of height1 and height2)

C++: `average = (height1 + height2)/2;`

Plain English: Print average to the screen

C++: `cout << "The average of 2 heights is " << average << "." << endl;`

Quick Review of basic C++ syntax

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

A directive and a header.

Gives us a lot of built-in code to use as tools in our applications, without having to re-implement all of those.

Like using a car without having to build its engine or steering.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

A namespace declarative.

A way to identify functions (tools) that have same name across different applications.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

THE main function.

The point from which code execution begins.

Without main, you can't get other functions to execute.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

A comment.

Totally ignored by the computer.

Just for our (humans – and AI) reference when we revisit our code after 10 years (or 1 week).

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

Declaration of variables.

Imagine them as baskets – ready to hold some values you give them later.

Double: a data type for storing decimal numbers (3.14, 2.7, etc.)

Other data types: int, char, float, etc.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

Input stream.

cin: character input.

Reads the keys you press on the keyboard and stores them in the respective variables.

Defined in the “iostream” library under the “std” namespace.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

An assignment.

Compute something on the right-hand-side.
Store its value in the left-hand-side variable.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

An output stream.
Send a sequence of strings
and variables to your
screen.

Quick Review of basic C++ syntax (ctd.)

▶ Run

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      /* Type your code here. */
7
8      double heightIronman, heightSpiderman;
9
10     cin >> heightIronman;
11     cin >> heightSpiderman;
12
13     double averageSuperHeroHeight = (heightIronman + heightSpiderman) / 2;
14
15     cout << "Average super hero height: " << averageSuperHeroHeight << endl;
16
17     return 0;
18 }
19
```

A return statement.

“0” means program successfully completed.
Non-zero: Something bad happened. Needs debugging!

Designing good test cases

- Test cases help to check if your program works in different situations
- The first few test cases should be straightforward
- Then try to find all possible ways the program can go wrong
- To decide on test cases:
 - Start with expected values
 - Move on to values that would “break” the program
- From average age example:
 - 17, 20 — straightforward, works
 - 10, 0 — works but doesn’t make sense
 - 0, 0 — works but doesn’t make sense
 - 15, -5 — works but doesn’t make sense
 - 10, 10.3 — depends on compiler, may not work or may take only what comes before “.”
 - hello, 5 — will fail because it expects an integer rather than string
- Later you will be asked to output error messages for some of these test cases

Any questions?

- Feel free to ask as many questions as you can!



Quick Recap

Sample code plan

1. Declare 2 variables for input and 1 variable for output. All “double” type as we have decimal numbers.
2. Get the height values from the user and store in the input variables.
3. Compute the average height and store in the output variable.
4. Print the output variable in the desired text format.

Sample test case

Input:

height1 = 5.5

height2 = 5.7

Expected Output:

Avg. height: 5.6

Note: You have to write down 3 test cases in total.
The output should have same format as requested.

Lab sheet tips

- Don't write line numbers for your code
- Feel free to use both sides of the sheet
- Try to write with medium-to-small font size to avoid an extra sheet
 - Though, I will provide extra sheet if needed... just ask :)

Overall program skeleton

```
#include <iostream>
using namespace std;
int main() {
    /* Your code here. */
    return 0;
}
```

Declaring variables

```
double height1, height2;
```

Taking inputs

```
cin >> height1;
cin >> height2;
```

Computing values

```
double averageHeight = (height1 + height2) / 2;
```

Printing outputs

```
cout << "Avg. height: ";
cout << averageHeight << endl;
```

Lab 1: House Prices

Sites like Zillow get input about house prices from a database and provide nice summaries for readers. Write a program with two inputs, current price and last month's price (both integers). Then, output a summary listing the price, the change since last month, and the estimated monthly mortgage computed as $(\text{currentPrice} * 0.045) / 12$.

Ex: If the input is 200000 210000, the output is:

```
This house is $200000. The change is $-10000 since last month.  
The estimated monthly mortgage is $750.
```

Step 1: Read the specs above and highlight any important information.

Step 2: Now, on your individual submission paper, write your group's code plan.

Step 3: Once you have the plan, now write the test cases that will be used to test your program for correctness and completeness. This should be done BEFORE you write any C++ code.

Step 4: Finally, translate your code plan above to a C++ program.

The cutting edge...

If you're willing to *really hone* your coding (algorithmic and problem solving) skills, register and participate in the following *really hard* programming contests:

1. CodeForces: <https://codeforces.com/>
2. ACM ICPC: <https://icpc.global/>



About ICPC

The International Collegiate Programming Contest is an algorithmic programming contest for college students. Teams of three, representing their university, work to solve the most real-world problems, fostering collaboration, creativity, innovation, and the ability to perform under pressure. Through training and competition, teams challenge each other to raise the bar on the possible. Quite simply, it is the oldest, largest, and most prestigious programming contest in the world.

[MORE ABOUT ICPC](#)



HOME TOP CATALOG CONTESTS GYM PROBLEMSET GROUPS RATING EDU API CALENDAR HELP

Current or upcoming contests

Name	Writers	Start	Length		
DeepInfra Round 1 (Codeforces Round, Div. 1 + Div. 2)		Oct/10/2026 08:05UTC-7	03:00	Before start 11 days	Before registration 11 days
Codeforces Round (Div. 1)		Oct/11/2026 11:35UTC-7	02:30	Before start 12 days	Before registration 12 days
Codeforces Round (Div. 2)		Oct/11/2026 11:35UTC-7	02:30	Before start 12 days	Before registration 12 days
Codeforces Round (Div. 1 + Div. 2)		Oct/17/2026 07:35UTC-7	03:00	Before start 3 weeks	Before registration 3 weeks

Past contests

Name	Writers	Start	Length		
Codeforces Round 1124 (Div. 1) Enter » Virtual participation »	Hamed_Ghaffari_ROOT eren_sweetweasel	Sep/26/2026 07:35UTC-7	02:30	Final standings	x803
Codeforces Round 1124 (Div. 2) Enter » Virtual participation »	Hamed_Ghaffari_ROOT eren_sweetweasel	Sep/26/2026 07:35UTC-7	02:30	Final standings	x13589